How K-Means Clustering Finds Patterns in Unlabelled Data

A Machine Learning Tutorial

Suthagar Thanushayan
Student ID-24173820
• 7PAM2002 Machine Learning and Neural Networks
University of Hertfordshire • March 2026
GitHub: https://github.com/thanushayan/kmeans_clustering_tutorial.git

1. Introduction

Most machine learning problems assume labelled data — the correct answer is known before training begins. But what if no labels exist? Unsupervised learning lets us discover hidden structure without any guidance. This tutorial demonstrates K-Means Clustering, one of the most widely used and interpretable unsupervised algorithms, applied to real mall customer data to identify distinct spending segments. K-Means belongs to the family of partition-based clustering methods and is particularly valued in industry because it scales efficiently to large datasets, produces interpretable results, and requires minimal configuration. It has found successful application in customer segmentation, image compression, document clustering, anomaly detection, and bioinformatics — making it an essential technique for any practitioner working with unlabelled data.

> By the end of this tutorial you will be able to: (1) explain how the K-Means algorithm converges step by step; (2) choose the optimal number of clusters K using the Elbow Method and Silhouette Score; (3) implement K-Means with scikit-learn in Python; (4) interpret cluster results for a real-world business problem; and (5) describe the key limitations of K-Means.

2. How K-Means Works

K-Means (MacQueen, 1967) partitions n data points into K clusters by minimising the Within-Cluster Sum of Squares (WCSS) — the total squared distance from each point to its assigned centroid:

```
WCSS = sum_{k=1}^{K} sum_{x_i in C_k} ||x_i - mu_k||^2
where mu_k is the mean (centroid) of cluster k
```

The algorithm (Lloyd, 1957) follows two steps, repeated until convergence:

- Assignment: assign every data point to the nearest centroid using Euclidean distance
- Update: move each centroid to the mean of all points currently assigned to it

Because centroids always move toward cluster means, the algorithm is guaranteed to converge — though it may settle at a local minimum rather than the global optimum. The quality of the final solution therefore depends heavily on the initial centroid placement. scikit-learn mitigates this by running n_init=10 random initialisations by default and keeping the result with the lowest WCSS. A more principled alternative is K-Means++ initialisation (Arthur & Vassilvitskii, 2007), which spreads initial centroids as far apart as possible, dramatically reducing the chance of poor convergence and is the default init strategy in scikit-learn. In practice, K-Means++ typically reaches a better solution in fewer iterations than purely random initialisation.

Figure 1 below visualises this convergence on a synthetic 3-cluster dataset. In the Init panel, centroids (stars) are placed randomly. By Iter 2, the clusters are already well-defined — demonstrating how quickly K-Means converges when clusters are well-separated.

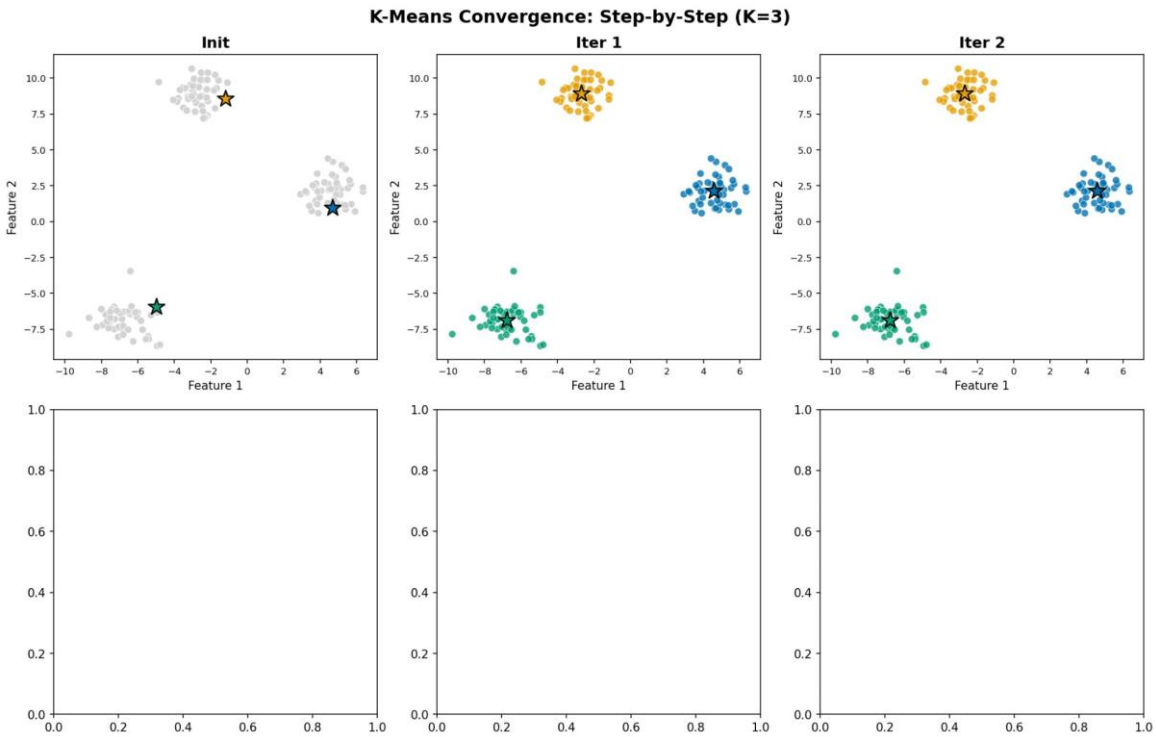

Figure 1: K-Means convergence step-by-step on a synthetic 3-cluster dataset. Stars = centroids. Points are colour-coded by cluster assignment.

3. Choosing the Right K

The most critical decision in K-Means is choosing K. Two complementary methods are recommended:

3.1 The Elbow Method

Plot the WCSS for K = 1 to 10. WCSS always decreases as K grows, but the rate of decrease slows sharply at the true number of clusters — forming a visual "elbow". Figure 2 shows this clearly at K=3 for the synthetic dataset.

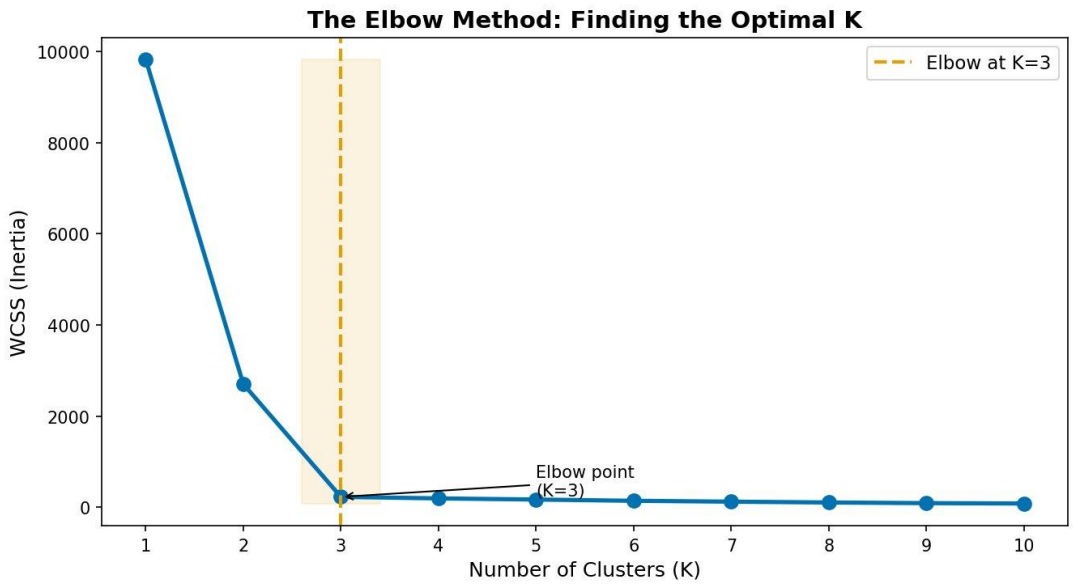

Figure 2: The Elbow Method. WCSS falls sharply until K=3, then levels off — indicating 3 as the optimal number of clusters for this dataset.

3.2 The Silhouette Score

The Silhouette Score (Rousseeuw, 1987) measures how well each point fits its assigned cluster compared to the nearest alternative. For each point i, it is computed as s(i) = (b(i) - a(i)) / max(a(i), b(i)), where a(i) is the

mean distance to all other points in the same cluster (cohesion) and b(i) is the mean distance to points in the nearest neighbouring cluster (separation). The overall score is the mean of s(i) across all points and ranges from -1 (wrong cluster) to +1 (well-clustered), with values near 0 indicating overlapping clusters. Crucially, unlike the Elbow Method, the Silhouette Score provides an absolute quality measure rather than a relative one, making it easier to compare solutions across different datasets. Figure 3 shows the score peaking at K=3, confirming the elbow result. The silhouette plot on the right shows all three clusters are well-separated with coefficients close to 1 and a mean of 0.86.

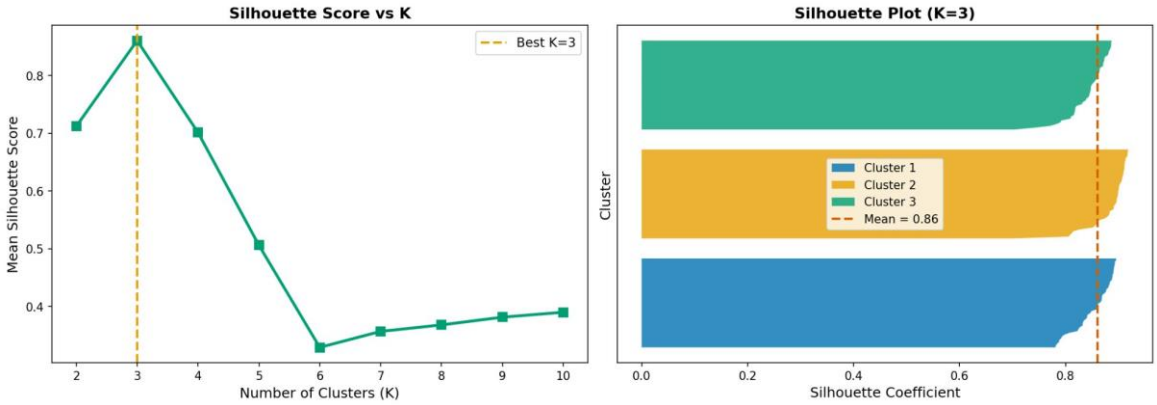

Figure 3: Left — Silhouette Score peaks at K=3. Right — Silhouette plot for K=3: all three clusters are well-defined with scores near 1.

4. Real-World Application: Mall Customer Segmentation

We apply K-Means to the Mall Customer Segmentation dataset (Kaggle, 2019) — 200 customers described by Annual Income (k$) and Spending Score (1–100). The goal is to identify distinct customer groups to inform targeted marketing strategies.

Applying both selection methods to the mall dataset confirms K=5 as optimal. Figure 4 shows the elbow at K=5 in the WCSS plot (left) and the silhouette score peaking at K=5 (right, score = 0.71).

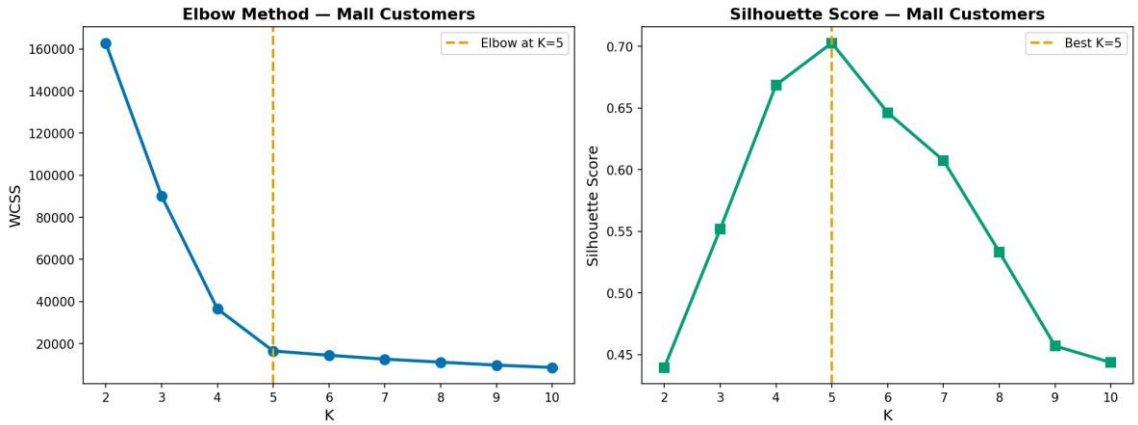

Figure 4: Both the Elbow Method and Silhouette Score confirm K=5 for the mall dataset.

With K=5, the algorithm identifies five actionable customer segments shown in Figure 5:

Segment	Profile	Recommended Strategy
Low Income, High Spend	Impulsive buyers, budget-stretched	Discounts, loyalty rewards
High Income, High Spend	Ideal customers, brand-loyal	Premium products, VIP service
Moderate All-Rounders	Balanced income and spending	Broad promotions, bundles
High Income, Low Spend	Careful savers, hard to convert	Value messaging, quality focus

Segment	Profile	Recommended Strategy
Low Income, Low Spend	Price-sensitive, budget-conscious	Sales events, clearance offers

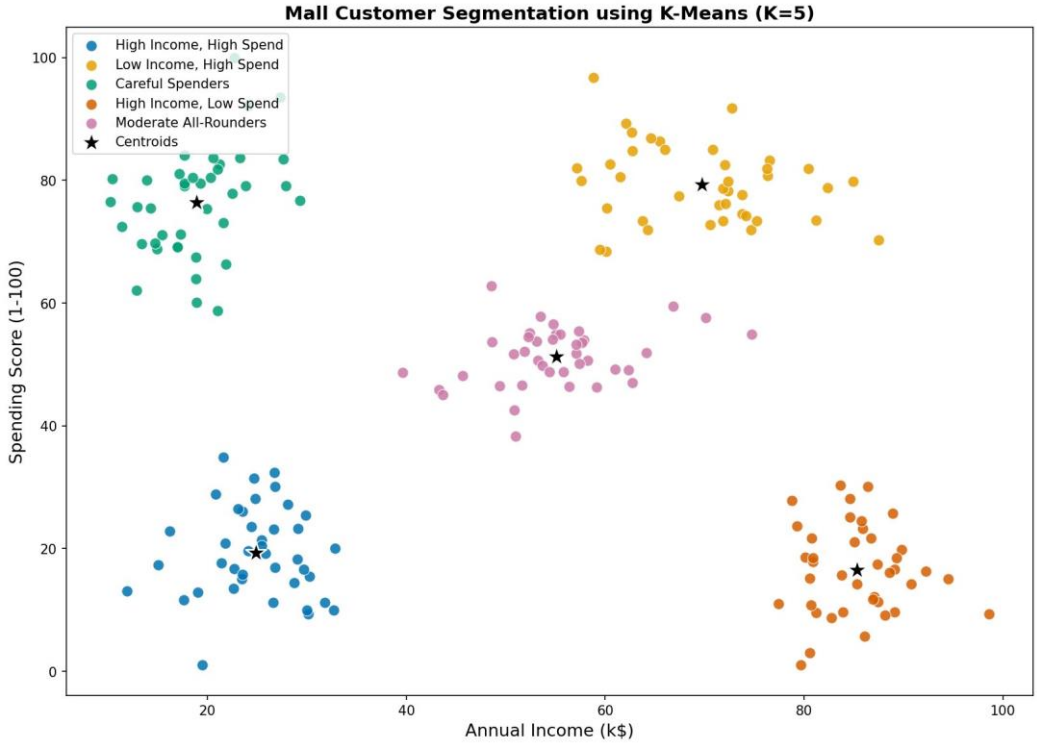

Figure 5: Five customer segments identified by K-Means (K=5). Each colour represents a distinct spending behaviour group. Black stars mark centroids.

Figure 6 shows the Voronoi decision boundaries — each coloured region contains all points closer to that centroid than any other. The clean separation confirms that K-Means is well-suited to this dataset.

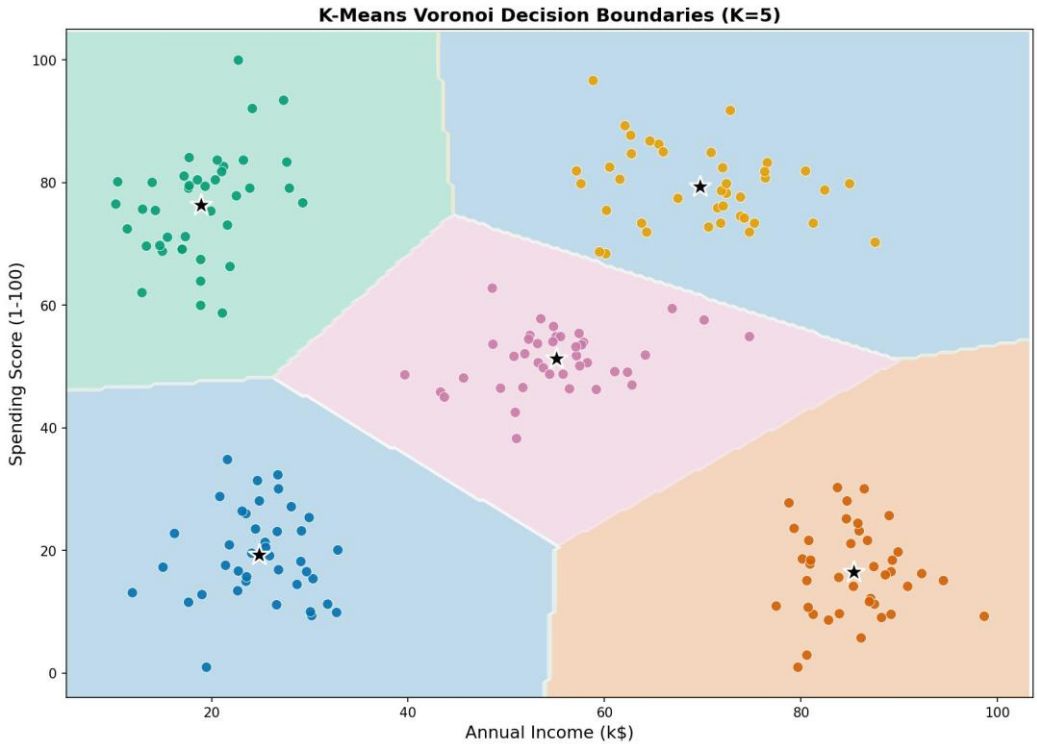

Figure 6: Voronoi decision boundaries show how K-Means partitions the feature space. Each region belongs to the nearest centroid.

5. Limitations of K-Means

Despite its strengths, K-Means has three well-known limitations, illustrated in Figure 7:

- Requires K in advance: you must specify K before running. A poor choice produces meaningless clusters. Always use the Elbow and Silhouette methods together.
- Assumes spherical clusters: K-Means uses Euclidean distance and struggles with crescent, ring, or elongated shapes. As Figure 7 (middle) shows, it incorrectly splits two interlocking moons into left and right halves rather than by shape. For non-spherical data, DBSCAN or Gaussian Mixture Models are better alternatives.
- Sensitive to outliers: extreme values pull centroids away from true cluster centres, distorting all assignments (Figure 7, right). Consider removing outliers first, or using K-Medoids as a more robust variant.

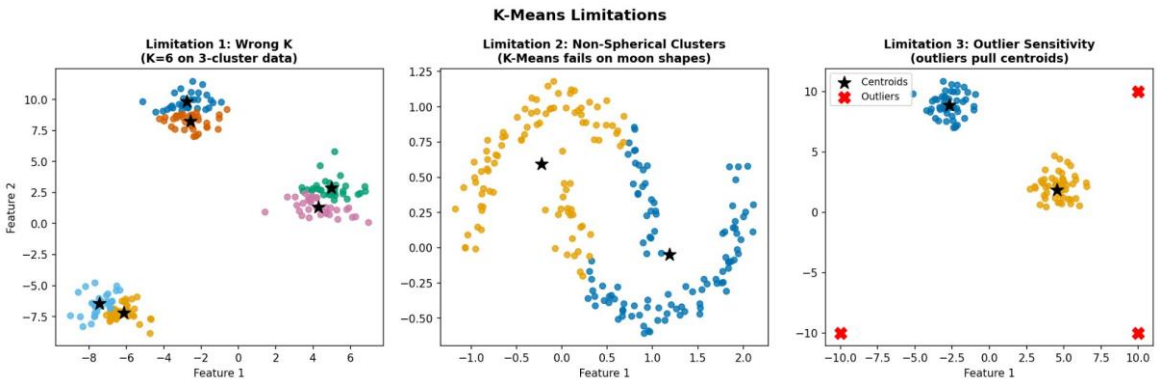

Figure 7: Three limitations of K-Means. Left: wrong K over-segments data. Middle: non-spherical clusters are misclassified. Right: outliers distort centroid positions.

6. Conclusion

K-Means is a powerful, scalable, and interpretable unsupervised algorithm. Its two-step iterative process — assign and update — reliably partitions data into meaningful groups with no labels required. By choosing K carefully using the Elbow Method and Silhouette Score, and understanding when the spherical-cluster assumption holds, K-Means can extract genuinely actionable insights from raw data, as demonstrated by the five distinct customer segments identified from the mall dataset. The algorithm runs in O(nKdI) time, where n is the number of points, K is the cluster count, d is the number of dimensions, and I is the number of iterations — making it practical even for datasets with millions of records. When K-Means falls short due to non-spherical geometry or high sensitivity to outliers, extensions such as DBSCAN, Gaussian Mixture Models, or K-Medoids offer robust alternatives. Nevertheless, K-Means remains the natural first choice for exploratory clustering: fast to implement, easy to interpret, and widely supported across every major machine learning library.

> Take-home message: K-Means iteratively assigns points to their nearest centroid and moves centroids to the cluster mean. Two steps, repeated until convergence, are enough to discover hidden structure in unlabelled data — with no labels required.